01

Data Science ve Pandas

Odak doğrudan karmaşık modellere geçmeden önce veriyi tanımak, kalite kontrolü yapmak ve temel veri manipülasyonu alışkanlığı kazanmaktadır.

Çalışmalar Kaggle kaynakları ve Titanic veri seti üzerinden yapıldı.

Ne Anladım? Bir makine öğrenmesi modelinin başarısı doğrudan verinin temizliğine ve kalitesine bağlıdır. Bu nedenle çalışmaya başlamadan önce verinin boyutunu, eksikliklerini ve dağılımını inceledim.



Veri Yükleme ve Boyut İncelemesi

```python
import pandas as pd

#colab'e yüklediğim titanic verisinin csv dosyası okunuyor
#Passenger Id sütununu satır indeksi yapıyoruz (index_col ='PassengeId)

dosya_adi = "train_and_test2.csv"
df = pd.read_csv(dosya_adi, index_col="Passengerid")

#Burada csv dosytasını pandas dataframe'e aktardım
#satırlar yolcuları,sütunlar is yolculara ait özellikleri tutuyor

#satır sütun sayısını verir
print("Veri seti boyutu (Satır,Sütun):" , df.shape)

#ilk 5 satırı inceler
print("İlk 5 Gözlem:")
display(df.head())
```

```
Veri seti boyutu (Satır,Sütun): (1309, 27)
İlk 5 Gözlem:
```

Passengerid	Age	Fare	Sex	sibsp	zero	zero.1	zero.2	zero.3	zero.4	zero.5	...	zero.12	zero.13	zero.14	Pclass	zero.15	zero.16
1	22.0	7.2500	0	1	0	0	0	0	0	0	...	0	0	0	3	0	0
2	38.0	71.2833	1	1	0	0	0	0	0	0	...	0	0	0	1	0	0
3	26.0	7.9250	1	0	0	0	0	0	0	0	...	0	0	0	3	0	0
4	35.0	53.1000	1	1	0	0	0	0	0	0	...	0	0	0	1	0	0
5	35.0	8.0500	0	0	0	0	0	0	0	0	...	0	0	0	3	0	0

5 rows × 27 columns

Veri seti Google Colab ortamına aktarılırken Passengerid sütununun satır indeksi olarak atandığı ve veri setinin 1309 satır ve 27 sütundan meydana geldiği görülmüştür.

Veri setini hazırlarken öncelikle indeks ataması yaparak veri tablosundaki gereksiz sütun kalabalığından kurtulduk. Ardından head() fonksiyonuyla ilk 5 satıra göz atarak sütun mimarisini net bir şekilde doğruladık. Bu ilk inceleme, veri setinde modelleme öncesi elenmesi gereken 'zero' gibi fazlalık sütunların bulunduğunu ortaya koydu.

Veri Tipleri ve Eksik Veri Analizi


```
#her sütunun veri tipini topluca kontrol eder
print("Sütun Veri Tipleri")
print(df.dtypes)

#her sütundaki eksik değer saysını hesaplar
print("\n Eksik Veri Sayısı")
eksik_sayisi=df.isnull().sum()
print(eksik_sayisi[eksik_sayisi>0])
#burada boşluklar bulunuyor. Boşluklar modelde kullanılamaz
#ya doldurulmalı ya da satır sütun bazında eleme yapılmalı
```

```
Sütun Veri Tipleri
Age          float64
Fare         float64
Sex            int64
sibsp          int64
zero           int64
zero.1         int64
zero.2         int64
zero.3         int64
zero.4         int64
zero.5         int64
zero.6         int64
Parch          int64
zero.7         int64
zero.8         int64
zero.9         int64
zero.10        int64
zero.11        int64
zero.12        int64
zero.13        int64
zero.14        int64
Pclass         int64
zero.15        int64
zero.16        int64
Embarked     float64
zero.17        int64
zero.18        int64
2urvived       int64
dtype: object

 Eksik Veri Sayısı
Embarked    2
dtype: int64
```

Kod sütunların float64 ve int64 tiplerinden oluştuğu. Eksik veri kontrolü sonucunda sadece Embarked sütununda 2 adet eksik değer kaldığını gösteriyor.

Makine öğrenmesi algoritmaları eksik verilerle (NaN) çalışamaz. Bu adımda verideki boşlukları tespit ederek modelleme öncesinde bu 2 hücre için bir temizleme veya doldurma kararı almak gerekiyor.

İstatistiksel Özetler ve Aykırı Değer Tespiti

```
#sayısal sütunların özet istatistiklerini hesaplar
print("\n Sayısal Değişken İstatistiksel Özeti ")
display(df.describe())
#çıktı yorumlama
#burada yaş ortalaması 29.503186
#Fare min 0, max 512.329200 çıkmış bu değer aykırı değer olabileceğini gösteriyor


# Kategorik değişkenlerdeki sınıfların sayımı
print("\n Cinsiyet Dağılımı")
print(df['Sex'].value_counts())


#tekrar etmeyenlerin listesi
print("\n Benzersizler")
print(df['Embarked'].unique())
```

Yolcuların yaş ortalaması 29.50, bilet ücreti ortalamasının 33.28 olduğu görülmekte. Bilet ücretinin (Fare) maksimum değerinin 512.33 çıktığı; cinsiyet dağılımında ise 843 erkek ve 466 kadın bulunmaktadır.

Sayısal Değişken İstatistiksel Özeti

	Age	Fare	Sex	sibsp	zero	zero.1	zero.2	zero.3	zero.4	zero.5	...	zero.12	zero.13	zero.14	P
count	1309.000000	1309.000000	1309.000000	1309.000000	1309.0	1309.0	1309.0	1309.0	1309.0	1309.0	...	1309.0	1309.0	1309.0	1309.0
mean	29.503186	33.281086	0.355997	0.498854	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	2.2
std	12.905241	51.741500	0.478997	1.041658	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.8
min	0.170000	0.000000	0.000000	0.000000	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	1.0
25%	22.000000	7.895800	0.000000	0.000000	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	2.0
50%	28.000000	14.454200	0.000000	0.000000	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	3.0
75%	35.000000	31.275000	1.000000	1.000000	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	3.0
max	80.000000	512.329200	1.000000	8.000000	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	3.0

8 rows × 27 columns

```
 Cinsiyet Dağılımı
Sex
0    843
1    466
Name: count, dtype: int64

 Benzersizler
[ 2.  0.  1. nan]
```

Windows'u Etkinleştir
Windows'u etkinleştirmek için Ayarlar'a gidin.

Bilet ücretindeki çeyrekliklere (%75'i 31.27) kıyasla maksimum değerin 512 olması, veride bariz aykırı değerlerin bulunduğunu gösteriyor. describe() çıktısı sayesinde tek tek satırlara bakmadan veri dağılımındaki uç noktaları okumayı görebiliriz.

Koşullu Seçim ve Filtreleme

```
# iloc ile ilk 5 satır ve ilk 3 sütunu konuma göre seçer
print("\n iloc ile ilk 5 satır ve 3 sütun")
display(df.iloc[:5, :3])

# loc ve Boolean Indexing ile çoklu şartlı filtreleme
#1.sınıfta olan ve hayatta kalan yolcuları listeler
kurtulan_birinci_sinif = df.loc[(df['Pclass'] == 1) & (df['2urvived'] ==1)]
print(f"\n1. Sınıf Hayatta Kalan Sayısı: {len(kurtulan_birinci_sinif)}")


# isin() kullanarak filtreleme
liman_cq = df.loc[df['Embarked'].isin([1, 2]), ['Age', 'Fare']]
print("\n C ve Q Limanından Binen İlk 3 Yolcu")
display(liman_cq.head(3))
```

loc ile konuma göre ilk 5 satır 3 sütunun çekilmesi. loc ve boolean sorgu ile 1. sınıfta olup hayatta kalan yolcuların filtrelenmesi (136 kişi). isin() ile belirli limanlardan binenlerin seçilmesi uygulamaları bulunuyor.

```
 iloc ile ilk 5 satır ve 3 sütun
```

Passengerid	Age	Fare	Sex
1	22.0	7.2500	0
2	38.0	71.2833	1
3	26.0	7.9250	1
4	35.0	53.1000	1
5	35.0	8.0500	0

```
1. Sınıf Hayatta Kalan Sayısı: 136

 C ve Q Limanından Binen İlk 3 Yolcu
```

Passengerid	Age	Fare
1	22.0	7.250
3	26.0	7.925
4	35.0	53.100

VERİ TEMİZLEME VE DÖNÜŞTÜRME

```
# Orijinal veriyi bozmamak için kopya üzerinde çalışıyoruz
islenmis_df =df.copy()

#yaş sütununundaki eskik değerleri yaşın medyanı ile dolduracağız
yas_medyani = islenmis_df['Age'].median()
islenmis_df['Age'] = islenmis_df['Age'].fillna(yas_medyani)

# Kabin sütunundaki eksik değerlere 'Unknown' atıyoruz
islenmis_df['Parch'] =islenmis_df['Parch'].fillna("Unkown")

# Kategorik değerleri Türkçeleştirme
islenmis_df['Sex'] = islenmis_df['Sex'].replace({'female':'Kadin','male':'Erkek'})

#Sınıf sütununu sayısal değil de bir kategori/etiket olarak algılaması için string yapıyoruz
islenmis_df['Pclass'] = islenmis_df['Pclass'].astype(str)
```

Kodlarda yaş sütunundaki boşlukların medyan ile doldurulması. Cinsiyet verilerinin string formatında düzenlenmesi. Bilet ücretinin ortalamadan çıkarılarak merkezileştirilmesi (Centered_Fare). apply() fonksiyonuyla yaş ve cinsiyete göre yeni bir Yolcu_Grubu sütunu üretilmesi (1155 Yetişkin Erkek, 154 Çocuk) anlatılıyor.

HARİTALAMA VE MERKEZİLEŞTİRME

```
#bilet ücretlerinin (Fare) genel aritmetik ortalamasını hesaplıyoruz
ortalama_ucret = islenmis_df['Fare'].mean()
#bilet ücretinden ortalama ücreti çıkarıyoruz
islenmis_df['Centered_Fare'] = islenmis_df['Fare'] - ortalama_ucret


#yaş ve cinsiyete göre yolcunun hangi grupta yer alacağını belirleyen fonksiyon
def yolcu_grubu(satir):
  if satir ['Age'] < 18:
    return 'Cocuk'
  elif satir ['Sex'] =='Kadin':
    return 'Yetiskin_Kadin'
  else:
    return 'Yetiskin_Erkek'

#yolcu_grubu fonkisyonunu DataFrame in tüm satırlarına uygular
#axis='columns satır satır çalışması ve sütun değerlerine erişim sağlar
islenmis_df['Yolcu_Grubu'] = islenmis_df.apply (yolcu_grubu, axis='columns')

print("\n Yeni Yolcu Grubu Dağılımı")
print(islenmis_df['Yolcu_Grubu'].value_counts())
#value_counts(), sütundaki benzersiz değerlerin kaç kez tekrarlandığını söyler
```

```
 Yeni Yolcu Grubu Dağılımı
Yolcu_Grubu
Yetiskin_Erkek    1155
Cocuk              154
Name: count, dtype: int64
```

Feature / Target Ayrımı ve Model Öncesi Son Kontrol


```
#değişken belirleniyor
# modelin tahmin etmesini istediğimiz bağımlı değişkeni seçiyoruz
# 'Survived' sütunu yolcunun hayatta kalıp kalmadığını (0: Öldü, 1: Kurtuldu) temsil ediyoruz
y = islenmis_df['2urvived']

#girdiler seçiliyor
#modeli yolcunun durumunu tahmin etmek için kullanacağımız bağımsız değişkenleri seçiyoruz
#katkısı olmayan sütunları buraya dahil etmiyoruz
X = islenmis_df[['Pclass','Sex','sibsp','Parch','Fare','Embarked', 'Yolcu_Grubu']]


print("\n -Model Öcesi Kontrol-")

#1-Hedef değişkenin sınıf dağılımı
# normalize=True kullanarak hayatta kalanlar ile hayatını kaybedenlerin oranını yüzde olarak görüyoruz
#amacı veri setinde sınıf dengesizliği olup olmadığını anlamak
#örneğin verinin %99'u 0 olsaydı model sadece 0 tahmin ederek yanıltıcı yüksek başarı gösterebilirdi
print("y Dağılımı (sınıf dengesi kontrolü):")
print(y.value_counts(normalize=True))

#2-Girdi matrisinin x boyutu
# X.shape çıktısı satır sayısı, sütun sayısı formatında gelir
#amacı seçilen girdi sayısını doğrulamak ve satır kaybı olup olmadığını kontrol etmektir
print(f"\n2. Girdi Matrisi X Boyutu: {X.shape} ")


#3-eksik veri kontrolü
# X.isnull().sum() her sütundaki eksik değer sayısını verir,
#ikinci .sum() ise tüm tablodaki toplam eksik hücreyi sayar
#amacı birçok makine öğrenmesi algoritması NaN değerlerle çalışamaz
#çıktının 0 olması ve modelleme öncesi eksik veri kalmadığını garantiler
print("3. Kalan Eksik Veri Var Mı?: ", X.isnull().sum().sum())
```

```
 -Model Öcesi Kontrol-
y Dağılımı (sınıf dengesi kontrolü):
2urvived
0    0.738732
1    0.261268
Name: proportion, dtype: float64

2. Girdi Matrisi X Boyutu: (1309, 7)
3. Kalan Eksik Veri Var Mı?:  2
```

Bağımlı değişken olarak 2urvived (y), bağımsız girdi özellikleri olarak 7 sütunluk x matrisi oluşturuldu. Hayatta kalma oranının %26.13, hayatını kaybetme oranının %73.87 olduğu görüldü. x matrisinin boyutunun (1309, 7) olduğu ve 2 adet eksik veri barındırdığı doğrulandı.

Sınıf dağılımına bakıldığında ciddi bir dengesizlik bulunmakta. Bu kontrol yapılmasaydı modelin ezber yapması veya yanıltıcı başarı üretmesi fark edilemezdi. Ayrıca model fonksiyonuna girmeden önce eksik hücre kontrolü yapmak zorunlu olarak test edilmeli.

Kontrol Sorusu:

Bir Veri Setinde Model Kurmadan Önce Neleri Kontrol Etmeliyiz?

1- Boyut ve veri yapısı kontrolü.

2-Veri tipleri ve uygun kategori dönüşümleri.

3-Eksik veri ve temizleme stratejisi.

4-Özet istatistikler ve aykırı değer tespiti.

5-Hedef değişken sınıf dengesi (Target balance).

Girdi matrisimizde Embarked sütunundan kalan 2 eksik veriyi satır bazında silmek mi yoksa en sık geçen değerle (mod) doldurmak mı bu veri seti için daha sağlıklıdır?

4. HAFTA

–28.10.2026–

TEŞEKKÜRLER

İlayda Bayrak